



TECNOLÓGICO
NACIONAL DE MÉXICO



**TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO SUPERIOR DE
MISANTLA**

ASIGNATURA: TALLER DE BASE DE DATOS

TITULAR: DR. IRAHAN OTONIEL JOSE GUZMAN

NOMBRE DE LA ACTIVIDAD:

CRUD

UNIDAD No.: UNIDAD II

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

SEMESTRE: 5 GRUPO: 503^a

PRESENTA:

GONZÁLEZ CONTRERAS MEREDYT CAROL -242T0076

FECHA DE ENTREGA: 16-09-2026

Índice

Índice de Figuras.....	3
Introducción	4
Restaurante.....	5
3.1 Preparación del entorno	5
3.2 Creación de la tabla comida.....	5
3.3 Alta de platillos	6
3.4 Modificación de precios.....	7
3.5 Baja de un platillo	7
Escuela	8
4.1 Creación de la tabla alumnos	8
4.2 Registro de estudiantes	8
4.3 Actualización de promedio	9
4.4 Baja de un estudiante	9
Tienda escolar	11
5.1 Creación de la tabla productos	11
5.2 Alta de productos	11
5.3 Actualización de existencia.....	12
5.4 Baja de un producto	12
Conclusión	14

Índice de Figuras

Figura 1 Preparación de la conexión a la base de datos y funciones auxiliares.....	5
Figura 2 Creación de la tabla comida.	6
Figura 3 Aumento del 10% al precio de los platos fuertes.	7
Figura 4 Eliminación de los postres del menú.	7
Figura 5 Creación de la tabla alumnos.....	8
Figura 6 Inserción de alumnos y verificación con SELECT.	9
Figura 7 Ajuste de promedio a los alumnos de 3ro A.....	9
Figura 8 Baja del alumno Carlos Ruiz.....	10
Figura 9 Creación de la tabla productos.	11
Figura 10 Inserción de productos y verificación con SELECT.	12
Figura 11 Descuento de existencia por venta de chocolates.....	12
Figura 12 Baja del producto jugo de manzana.....	13

Introducción

Esta actividad consistió en practicar el uso del lenguaje SQL dentro de un entorno de bases de datos relacionales, usando SQLite como motor y Google Colab como espacio de trabajo. A través de Python y las bibliotecas sqlite3 y pandas se crearon tablas, se les cargó información y se les aplicaron cambios, cubriendo así las cuatro operaciones que forman el ciclo CRUD: crear, leer, actualizar y borrar registros.

Para ponerlo en práctica se plantearon tres escenarios distintos: el menú de un restaurante, el control de alumnos de una escuela y el inventario de una tienda escolar. En cada uno se pensó una tabla adecuada al contexto, tomando en cuenta qué campos necesitaba, cuáles no podían quedar vacíos y qué valores debían tener por defecto.

El interés de la actividad estuvo en experimentar de primera mano cómo se comporta una base de datos ante cada tipo de instrucción, y en aprender a leer los mensajes de error que van apareciendo en el camino para corregirlos.

A continuación se muestra el desarrollo de los tres ejercicios, con el código empleado y capturas de su ejecución en Colab.

Link de la plataforma Colab: <https://colab.research.google.com/drive/1vuvWo22VZXaPELtRj-z7X2AcgnT0fWYP?usp=sharing>

Restaurante

3.1 Preparación del entorno

Antes de trabajar con cualquiera de las tres tablas se dejó lista la conexión a la base de datos: se importaron sqlite3 y pandas, se abrió una base en memoria y se definieron dos funciones de apoyo, una para correr consultas SELECT y mostrarlas como tabla (query) y otra para ejecutar INSERT, UPDATE, DELETE o cualquier instrucción DDL (run).

```
import sqlite3
import pandas as pd

# Conexión en memoria (se pierde al cerrar el notebook)
conn = sqlite3.connect(":restaurant.db:")
cur = conn.cursor() #objeto para ejecutar comandos de SQL

def query(sql, params=()):
    """Ejecuta un SELECT y devuelve un DataFrame para verlo bonito."""
    return pd.read_sql_query(sql, conn, params=params)

def run(sql, params=()):
    """Ejecuta INSERT/UPDATE/DELETE/DDL y hace commit."""
    cur.execute(sql, params)
    conn.commit()
    print("OK ->", sql.strip().splitlines()[0], "...")
```

Figura 1 Preparación de la conexión a la base de datos y funciones auxiliares.

3.2 Creación de la tabla comida

La tabla comida guarda el menú del restaurante. Tiene un id_platillo como llave primaria, un nombre obligatorio, una categoria para agrupar los platillos (entrada, plato fuerte, postre, bebida), un precio también obligatorio y un campo disponible de tipo booleano que por defecto queda en TRUE.

Ejercicio 1

Una cadena de restaurantes necesita registrar en su base de datos el menú de platillos que ofrece. Crea una tabla llamada comida que cumpla con las siguientes especificaciones:

[]

```
cur.executescript('''
DROP TABLE IF EXISTS comida;

CREATE TABLE comida (
    id_platillo INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    categoria VARCHAR(50),
    precio DECIMAL(6,2) NOT NULL,
    disponible BOOLEAN DEFAULT TRUE
);
''')
conn.commit()
print("Tabla recreada con columna 'disponible' correcta")
```

✓ ... Tabla recreada con columna 'disponible' correcta

Figura 2 Creación de la tabla comida.

3.3 Alta de platillos

Se dieron de alta tres platillos (tacos al pastor, ensalada César y flan napolitan) indicando su categoría, precio y disponibilidad, y se confirmó con un SELECT que quedaran registrados correctamente.

OK -> INSERT INTO comida (id_platillo, nombre, categoria, precio, disponible) VALUES ...

[]

```
# INSERT INTO

run('''
INSERT INTO comida (id_platillo, nombre, categoria, precio, disponible) VALUES
    (1, 'Tacos al pastor', 'plato fuerte', 85.00, 1),
    (2, 'Ensalada César', 'entrada', 65.50, 1),
    (3, 'Flan napolitano', 'postre', 45.00, 1);
''')

query("SELECT * FROM comida")
```

✓ ... OK -> INSERT INTO comida (id_platillo, nombre, categoria, precio, disponible) VALUES ...

	id_platillo	nombre	categoria	precio	disponible
0	1	Tacos al pastor	plato fuerte	85.0	1
1	2	Ensalada César	entrada	65.5	1
2	3	Flan napolitano	postre	45.0	1

Inserción de platillos y verificación con SELECT.

3.4 Modificación de precios

Con UPDATE se subió un 10% el precio de todos los platillos de la categoría plato fuerte, usando SET para el nuevo cálculo y WHERE para acotar el cambio solo a esa categoría.

```
# UPDATE

run("UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'plato fuerte';")

query("SELECT id_platillo, nombre, precio FROM comida WHERE categoria = 'plato fuerte'")

... OK -> UPDATE comida SET precio = precio * 1.10 WHERE categoria = 'plato fuerte'; ...
```

id_platillo	nombre	precio
0	1 Tacos al pastor	93.5

Figura 3 Aumento del 10% al precio de los platos fuertes.

3.5 Baja de un platillo

Se retiraron del menú los platillos de categoría postre mediante DELETE, filtrando con WHERE para no afectar el resto de los registros.

```
# DELETE

run("DELETE FROM comida WHERE categoria = 'postre';")

query("SELECT * FROM comida")

... OK -> DELETE FROM comida WHERE categoria = 'postre'; ...
```

id_platillo	nombre	categoria	precio	disponible
0	1 Tacos al pastor	plato fuerte	93.5	1
1	2 Ensalada César	entrada	65.5	1

Figura 4 Eliminación de los postres del menú.

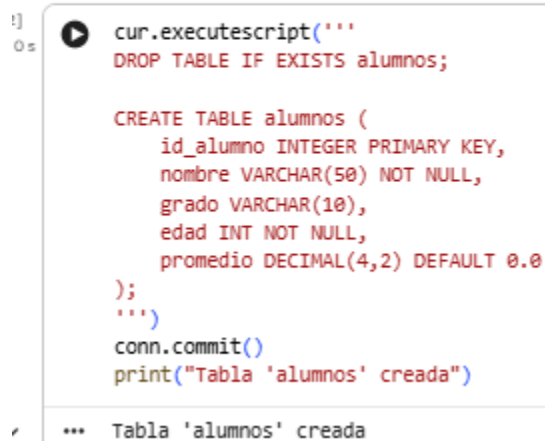
Escuela

4.1 Creación de la tabla alumnos

Para el registro escolar se definió la tabla alumnos con id_alumno como llave primaria, nombre y edad obligatorios, grado para indicar el grupo del estudiante y promedio con un valor inicial de 0.0.

Ejemplo 2 — Escuela (tabla alumnos)

Una escuela quiere registrar la información básica de sus alumnos.



```
cur.executescript('''
DROP TABLE IF EXISTS alumnos;

CREATE TABLE alumnos (
    id_alumno INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    grado VARCHAR(10),
    edad INT NOT NULL,
    promedio DECIMAL(4,2) DEFAULT 0.0
);
''')
conn.commit()
print("Tabla 'alumnos' creada")

... Tabla 'alumnos' creada
```

Figura 5 Creación de la tabla alumnos

4.2 Registro de estudiantes

Se cargaron tres alumnos (Ana López, Carlos Ruiz y María Sánchez) con su grado, edad y promedio, y se listó la tabla para confirmar que los tres quedaran dados de alta.

```
# INSERT INTO
run("""
INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES
    ('Ana López', '3ro A', 9, 8.5),
    ('Carlos Ruiz', '5to B', 11, 7.8),
    ('María Sánchez', '3ro A', 8, 9.2);
""")

query("SELECT * FROM alumnos")
```

... OK -> INSERT INTO alumnos (nombre, grado, edad, promedio) VALUES ...

	id_alumno	nombre	grado	edad	promedio
0	1	Ana López	3ro A	9	8.5
1	2	Carlos Ruiz	5to B	11	7.8
2	3	María Sánchez	3ro A	8	9.2

Figura 6 Inserción de alumnos y verificación con SELECT.

4.3 Actualización de promedio

Se aplicó un UPDATE para subir 0.3 puntos el promedio de los alumnos del grado 3ro A, mostrando cómo una misma instrucción puede afectar a varios registros a la vez cuando la condición del WHERE se cumple para más de uno.

```
# UPDATE
run("UPDATE alumnos SET promedio = promedio + 0.3 WHERE grado = '3ro A';")

query("SELECT id_alumno, nombre, grado, promedio FROM alumnos WHERE grado = '3ro A'")
```

... OK -> UPDATE alumnos SET promedio = promedio + 0.3 WHERE grado = '3ro A'; ...

	id_alumno	nombre	grado	promedio
0	1	Ana López	3ro A	8.8
1	3	María Sánchez	3ro A	9.5

Figura 7 Ajuste de promedio a los alumnos de 3ro A.

4.4 Baja de un estudiante

Se dio de baja al alumno Carlos Ruiz con DELETE, filtrando por su nombre para eliminar únicamente ese registro y dejar intacto al resto del grupo.

```
}  
0s  
# DELETE  
run("DELETE FROM alumnos WHERE nombre = 'Carlos Ruiz';")  
  
query("SELECT * FROM alumnos")  
  
... OK -> DELETE FROM alumnos WHERE nombre = 'Carlos Ruiz'; ...  
id_alumno nombre grado edad promedio  
0 1 Ana López 3ro A 9 8.8  
1 3 María Sánchez 3ro A 8 9.5
```

Figura 8 Baja del alumno Carlos Ruiz.

Tienda escolar

5.1 Creación de la tabla productos

Para el inventario de la tiendita se creó la tabla productos, con id_producto como llave primaria, nombre y precio obligatorios, tipo para clasificar el artículo (snack, bebida, dulce) y existencia con valor inicial 0 para llevar el conteo en almacén.

Ejemplo 3 — Tiendita escolar

La tiendita de la escuela necesita llevar el control de los productos que vende.

```
[6] cur.executescript('''
✓ Os DROP TABLE IF EXISTS productos;

CREATE TABLE productos (
    id_producto INTEGER PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30),
    precio DECIMAL(6,2) NOT NULL,
    existencia INT DEFAULT 0
);
''')
conn.commit()
print("Tabla 'productos' creada")
```

... Tabla 'productos' creada

Figura 9 Creación de la tabla productos.

5.2 Alta de productos

Se registraron tres artículos (papas fritas, jugo de manzana y chocolate) con su tipo, precio y existencia inicial, y se consultó la tabla para verificar el inventario cargado.

```
1 0s # INSERT INTO
run('
INSERT INTO productos (nombre, tipo, precio, existencia) VALUES
  ('Papas fritas', 'snack', 15.00, 30),
  ('Jugo de manzana', 'bebida', 12.50, 25),
  ('Chocolate', 'dulce', 10.00, 40);
')

query("SELECT * FROM productos")

... OK -> INSERT INTO productos (nombre, tipo, precio, existencia) VALUES ...
  id_producto  nombre  tipo  precio  existencia
0            1  Papas fritas  snack    15.0      30
1            2  Jugo de manzana  bebida    12.5      25
2            3    Chocolate    dulce    10.0      40
```

Figura 10 Inserción de productos y verificación con SELECT.

5.3 Actualización de existencia

Tras simular la venta de varias unidades, se usó UPDATE para descontar 10 piezas de la existencia del chocolate, dejando ver cómo se refleja una operación de inventario en la base de datos.

```
[8] 0s # UPDATE
run("UPDATE productos SET existencia = existencia - 10 WHERE nombre = 'Chocolate';")

query("SELECT id_producto, nombre, existencia FROM productos WHERE nombre = 'Chocolate'")

OK -> UPDATE productos SET existencia = existencia - 10 WHERE nombre = 'Chocolate'; ...
  id_producto  nombre  existencia
0            3  Chocolate      30
```

Figura 11 Descuento de existencia por venta de chocolates.

5.4 Baja de un producto

Finalmente, se eliminó el jugo de manzana del catálogo con DELETE, ya que dejó de venderse en la tienda, confirmando el cambio con una consulta posterior.

```
[9]
✓ 0s # DELETE
run("DELETE FROM productos WHERE nombre = 'Jugo de manzana';")

query("SELECT * FROM productos")

... OK -> DELETE FROM productos WHERE nombre = 'Jugo de manzana'; ...
```

	id_producto	nombre	tipo	precio	existencia
0	1	Papas fritas	snack	15	30
1	3	Chocolate	dulce	10	30

Figura 12 Baja del producto jugo de manzana.

Conclusión

Trabajar los tres ejercicios de manera consecutiva ayudó a ver que, sin importar el tema de la tabla, la secuencia de trabajo con SQL es prácticamente la misma: primero se piensa la estructura y sus restricciones, después se cargan datos de prueba, luego se modifican bajo alguna condición y al final se depuran los registros que ya no aplican.

En el camino surgieron errores típicos de quien está aprendiendo el lenguaje (comas de más, palabras clave mal escritas, referencias a columnas o tablas que no coincidían con lo ya creado, o intentos de repetir una llave primaria) que terminaron siendo útiles para entender mejor por qué ciertas reglas de SQL existen, en particular el papel de WHERE para no afectar registros de más.

En general, la práctica dejó una base sólida sobre el manejo de datos dentro de una tabla, que servirá como punto de partida para temas que normalmente siguen después, como relacionar varias tablas entre sí mediante llaves foráneas.